LANGNATION CLOUD PLATFORM

Final Capstone Project Report

Secure, Highly Available, Containerized Learning Platform with AI/RAG on AWS

Author	Seraj Ahmad Siddiqui
Course	AWS Cloud Computing with AI Bootcamp
Training Provider	neue fische
Project	LangNation Cloud Platform
Environment	Development / disposable training sandbox
AWS Region	us-east-1
Submission	Final capstone submission - October 2026

Public endpoint: https://cloud.namienterprises.in

Repository: github.com/sheikhseraj/langnation-cloud-platform

Guidance: Micheal (Theory Teacher) | Rahul Patel (Practical Teacher) | Dominic Singaraj (Head of Department)

Declaration of Original Work

I declare that this report documents the LangNation Cloud Platform capstone work completed during the AWS Cloud Computing with AI Bootcamp. The architecture, implementation decisions, validation results, troubleshooting records and operational procedures described here correspond to the project work carried out across Milestones 1-12. External product documentation and platform references are acknowledged in the References section.

The report has been prepared as a final technical submission. Where the training environment imposed limitations - including nightly sandbox cleanup and restricted AWS permissions - those limitations are stated explicitly rather than being presented as production capabilities.

Signature	**Date**
_____	_____

Abstract

LangNation Cloud Platform is a capstone project that transforms a learning application into a secure, highly available, reproducible and AI-enabled AWS platform. The project was implemented incrementally across twelve milestones. The final architecture uses a two-Availability-Zone Amazon VPC, public load-balancing and egress subnets, private application subnets running Amazon ECS on AWS Fargate, and private database subnets hosting Amazon RDS for MariaDB. Amazon CloudFront, AWS Certificate Manager and AWS WAF provide the protected HTTPS edge, while Amazon Cognito supplies managed user authentication. Infrastructure is defined in Terraform, with HCP Terraform used as the authoritative remote state and plan/apply system. GitHub Actions uses OpenID Connect (OIDC) and AWS STS for short-lived deployment credentials, avoiding long-lived AWS keys.

The later milestones extend the platform with event-driven document processing, a retrieval-augmented generation (RAG) pipeline and an operational AI tutor named Lina. Uploaded knowledge is routed through Amazon S3, EventBridge, SQS and AWS Lambda. Amazon Titan Text Embeddings V2 creates 1024-dimensional vectors, which are stored and queried with Amazon S3 Vectors. The real LangNation application embeds a user question, retrieves relevant LangNation context and invokes Claude Haiku 4.5 to produce grounded responses. Monitoring and governance are provided by Amazon CloudWatch dashboards and alarms together with a multi-Region AWS CloudTrail trail. CI/CD quality gates validate the application, Terraform configuration, container startup and container vulnerabilities before a release can proceed.

The project was validated through controlled tests rather than resource creation alone. Examples include ECS scaling, WAF blocking, Cognito JWT authorization, SQS retry and dead-letter behavior, semantic retrieval ranking, MariaDB TLS verification, CloudWatch alarm recovery, container vulnerability remediation and an ECS task-failure test in which the public endpoint remained healthy while the service recovered to two running tasks in approximately 44 seconds. AWS Cost Explorer was also used to review actual October usage, showing approximately $23.78 before tax and $28.07 including the displayed tax. The completed capstone demonstrates a connected cloud platform with explicit security boundaries, automated recovery, measurable resilience and evidence-based design trade-offs.

Keywords: AWS, Terraform, HCP Terraform, ECS Fargate, MariaDB, CloudFront, WAF, Cognito, EventBridge, SQS, Lambda, Bedrock, S3 Vectors, RAG, CI/CD, CloudWatch, CloudTrail, Resilience, Cost Optimization

Acknowledgements

I would like to thank the teaching and leadership team of the AWS Cloud Computing with AI Bootcamp for their guidance, feedback and practical support throughout the capstone. Particular thanks go to Micheal for theory guidance, Rahul Patel for practical guidance and Dominic Singaraj for departmental leadership. I also acknowledge the wider teaching team and classmates whose discussions, testing feedback and questions helped improve the project.

The training sandbox created an unusual but valuable engineering constraint: infrastructure was regularly removed overnight. Rather than treating this as only an inconvenience, the project used it as a real requirement for Infrastructure as Code, recovery automation, persistent DNS and reproducible deployment. This significantly strengthened the final design.

Table of Contents

Section	Title	Page
1	Introduction	8
2	Problem Statement, Scope and Objectives	9
3	Methodology and Project Environment	10
4	Final System Architecture	12
5	Milestones 1-3: Foundation, Containers and Availability	14
6	Milestones 4-6: CI/CD, Data, Edge Security and Authentication	15
7	Milestone 7: Serverless and Event-Driven Processing	17
8	Milestone 8: RAG Knowledge Pipeline	19
9	Milestone 9: Real Application, Database Restore and Lina RAG	20
10	Milestone 10: Monitoring, Security and Governance	22
11	Milestone 11: CI/CD Quality Gates and Release Hardening	24
12	Milestone 12: Resilience, Cost Optimization and Final Validation	26
13	OP01: Automated Morning Sandbox Recovery	30
14	Security Architecture and Threat Reduction	31
15	Testing, Validation and Evidence	32
16	Cost Analysis and Optimization Decisions	33
17	Discussion, Limitations and Lessons Learned	34
18	Conclusion and Future Work	35
	References	36
	Appendices	37

List of Figures

No.	Figure title
Figure 1	Final LangNation Cloud Platform architecture (M1-M12)
Figure 2	CI/CD deployment pipeline
Figure 3	Milestone 7 event-driven processing architecture
Figure 4	Milestone 9 application and Lina integration architecture
Figure 5	Milestone 10 monitoring and governance architecture
Figure 6	Milestone 11 CI/CD quality-gate architecture
Figure 7	M11 successful quality-gated deployment evidence
Figure 8	M12 resilience baseline evidence
Figure 9	M12 self-healing live monitoring evidence
Figure 10	M12 final replacement-task state
Figure 11	M12 October Cost Explorer evidence
Figure 12	M12 cost-optimization summary
Figure 13	OP01 automated morning sandbox recovery flow

List of Tables

No.	Table title
Table 1	Project objectives and success criteria
Table 2	Final technology stack
Table 3	Network addressing and placement
Table 4	Milestone completion summary
Table 5	M7 queue and Lambda configuration
Table 6	M8 RAG configuration
Table 7	M10 operational alarms
Table 8	M12 cost breakdown
Table 9	Security-control mapping
Table 10	Final validation matrix
Table 11	Major technical challenges and permanent fixes
Table 12	Final technical configuration

Abbreviations

Abbreviation	Meaning
ACM	AWS Certificate Manager
ALB	Application Load Balancer
API	Application Programming Interface
AZ	Availability Zone
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Delivery
DLQ	Dead Letter Queue
ECR	Elastic Container Registry
ECS	Elastic Container Service
HCP	HashiCorp Cloud Platform
IAM	Identity and Access Management
IaC	Infrastructure as Code
IGW	Internet Gateway
JWT	JSON Web Token
NAT	Network Address Translation
OIDC	OpenID Connect
RAG	Retrieval-Augmented Generation
RDS	Relational Database Service
S3	Simple Storage Service
SG	Security Group
SQS	Simple Queue Service
STS	Security Token Service
TLS	Transport Layer Security
VPC	Virtual Private Cloud
WAF	Web Application Firewall

1. Introduction

1.1 Background

Cloud platforms are often demonstrated as a sequence of isolated services: a virtual network, a container service, a database, a load balancer or an AI model. The engineering challenge is not creating those resources individually, but integrating them into a platform with clear trust boundaries, ownership rules, deployment automation, observability, resilience and recovery. The LangNation Cloud Platform capstone was designed around that integration problem.

LangNation is a language-learning platform. The capstone uses the LangNation use case to design an AWS environment capable of serving students and application users through a stable public endpoint while keeping application compute and relational data private. The project also extends the platform with document ingestion and retrieval-augmented generation so that an AI tutor can answer from LangNation knowledge rather than relying only on general model knowledge.

The project was implemented inside an educational AWS sandbox that is regularly cleaned. This made reproducibility a practical requirement. A design that depended on manually created resources would fail after each cleanup. The final solution therefore treats infrastructure, deployment, database restoration and DNS recovery as code-driven and repeatable processes.

1.2 Project Motivation

- Create a realistic cloud architecture rather than a single-service demonstration.
- Practice Infrastructure as Code and remote state management under an ephemeral environment constraint.
- Implement private container and database tiers with a controlled public edge.
- Use short-lived identities for CI/CD instead of storing permanent AWS access keys.
- Build a real asynchronous failure path using retries and a dead-letter queue.
- Integrate RAG using AWS-native services and prove semantic retrieval before answer generation.
- Add operational monitoring, audit logging and release quality gates.
- Validate resilience and analyze real cost data rather than only estimating theoretical behavior.

1.3 Report Structure

This report follows the project evolution from the initial network foundation through final resilience and cost validation. Chapters 5-12 describe the milestone implementation, while later chapters consolidate security, validation, cost, operational recovery and lessons learned. The appendices provide configuration, repository structure and validation matrices for technical review.

2. Problem Statement, Scope and Objectives

2.1 Problem Statement

The capstone required a platform that could remain secure and understandable while combining networking, container deployment, data persistence, authentication, asynchronous processing, AI retrieval, monitoring and automation. The disposable training environment added a second problem: resources could disappear overnight, including runtime endpoints and container registries. The solution therefore had to be not only deployable, but recoverable.

2.2 Scope

The project scope covers the AWS application platform and its operational automation in the development sandbox. It does not claim production-scale traffic, production compliance certification, global disaster recovery or zero-downtime database failover testing. The design deliberately uses production-style patterns - such as multi-AZ application capacity, private data placement, CloudFront/WAF edge protection and audit logging - while remaining within the constraints and cost profile of a bootcamp environment.

Table 1. Project objectives and success criteria

Objective	Implementation approach	Success criterion
Secure network foundation	Public, private application and private database tiers across two AZs	Private ECS/RDS placement; layered SG path
Containerized runtime	Run Node.js without managing EC2 hosts	ECS Fargate service behind ALB
High availability	Avoid single-task/single-AZ application dependency	Two baseline tasks across two AZs; ALB health routing
Automated deployment	Release repeatably without long-lived AWS keys	GitHub Actions + OIDC + STS + Git-SHA images
Secure data layer	Keep relational/file data private and encrypted	Multi-AZ MariaDB, Secrets Manager, private/versioned S3
Protected public edge	Stable HTTPS URL with web filtering and authentication	Route 53 + CloudFront + ACM + WAF + Cognito
Event processing	Handle document events asynchronously and durably	EventBridge + SQS + Lambda + DLQ
RAG knowledge layer	Create semantic retrieval over LangNation content	Titan embeddings + S3 Vectors
Grounded AI runtime	Answer using retrieved LangNation sources	Lina API + Claude Haiku + grounded=true validation
Operations/governance	Observe health and preserve audit evidence	CloudWatch dashboard/alarms + CloudTrail
Release hardening	Block invalid/vulnerable releases	Tests + Terraform validation + smoke test + Trivy
Resilience/cost validation	Prove self-healing and assess real spend	Controlled ECS failure + Cost Explorer analysis
Recovery automation	Restore disposable environment after activation	OP01 rebuild + DB restore + deploy + DNS sync + health

3. Methodology and Project Environment

3.1 Incremental Milestone Method

The project used an incremental architecture method. Each milestone introduced one new concern while retaining and validating the previous platform. This made troubleshooting easier and created an evidence trail. Networking was built before containers, containers before load balancing, the data layer before edge security, and event ingestion before RAG answer generation. Later milestones focused on monitoring, release quality, resilience and cost.

3.2 Learning-Oriented Engineering Process

1. Design the milestone and identify the AWS services and ownership boundaries.
2. Implement the required Terraform, application or workflow changes.
3. Run local formatting and validation checks.
4. Use HCP Terraform as the authoritative plan/apply/state system.
5. Validate actual runtime behavior using AWS CLI, health endpoints, logs and controlled tests.
6. Record errors, root causes, permanent fixes and lessons learned.
7. Update milestone documentation and architecture diagrams before moving forward.

3.3 Development Environment

Table 2. Final technology stack

Item	Final project value
Operating system	Windows 11
AWS Region	us-east-1
AWS sandbox account	Educational sandbox; disposable/nightly cleanup
AWS CLI	2.35.15
Terraform	1.16.2
AWS provider	6.67.0
Docker	29.7.2
Git	2.53.0
HCP Terraform org	cloudforge-capstone
HCP workspace	langnation-capstone-dev
Repository	LangNation-Cloud-Platform-Capstone / GitHub
Runtime	Node.js application on ECS Fargate
Final database	Amazon RDS for MariaDB 11.8.9
Public endpoint	https://cloud.namienterprises.in

3.4 Infrastructure Ownership Model

```
Local workstation
  -> terraform init / fmt / validate

HCP Terraform
  -> authoritative remote state
  -> plan / apply / rebuild
```

```
GitHub Actions
  -> quality gates
  -> Docker build / ECR push
  -> ECS application deployment
  -> database restore orchestration
  -> DNS recovery orchestration

Application Auto Scaling
  -> live ECS desired count

Persistent Route 53 account
  -> stable public DNS destination
```

Design principle: A single live value should have one clear owner. Terraform defines infrastructure, CI/CD owns live task-definition revisions, and Application Auto Scaling owns the live desired task count.

4. Final System Architecture

4.1 Architectural Overview

The final platform consists of a protected public edge, a two-AZ VPC, private ECS compute, a private Multi-AZ relational database, private object/vector storage, event-driven document processing, an AI retrieval/generation path, CI/CD automation, monitoring, audit logging and recovery automation. Route 53 is intentionally persistent outside the disposable training account so that the public hostname can survive sandbox recreation.

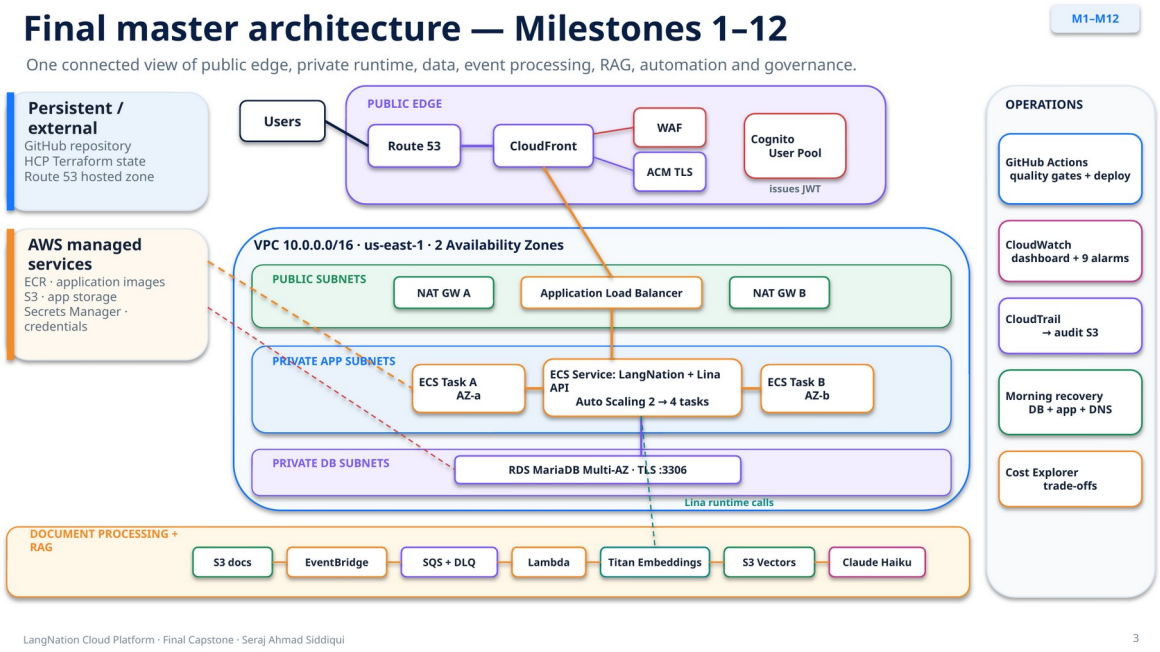


Figure 1. Final LangNation Cloud Platform architecture (Milestones 1-12).

4.2 Network Placement

Table 3. Network addressing and placement

Component	CIDR	Purpose
VPC	10.0.0.0/16	Whole platform network
Public subnet A	10.0.1.0/24	ALB/NAT in us-east-1a
Public subnet B	10.0.2.0/24	ALB/NAT in us-east-1b
Private app A	10.0.11.0/24	ECS tasks in us-east-1a
Private app B	10.0.12.0/24	ECS tasks in us-east-1b
Private DB A	10.0.21.0/24	RDS DB subnet in us-east-1a
Private DB B	10.0.22.0/24	RDS DB subnet in us-east-1b

4.3 Final Request Path

```
User browser
  -> Route 53
  -> CloudFront (HTTPS + ACM)
  -> AWS WAF
  -> ALB origin over HTTP :80
```

```
-> Target Group :3000
-> Private ECS Fargate task
-> LangNation application
    -> private MariaDB over verified TLS
    -> S3 / S3 Vectors / Bedrock as required
```

4.4 Milestone Completion Summary

Table 4. Milestone completion summary

Item	Scope	Status
M1	Foundation & Networking	Complete
M2	Docker, ECR & ECS Fargate	Complete
M3	ALB, Multi-AZ & Auto Scaling	Complete
M4	Automated Deployment	Complete
M5	Database & Storage	Complete
M6	Edge, DNS, Security & Authentication	Complete
M7	Serverless & Event-Driven Processing	Complete
M8	RAG Knowledge Pipeline	Complete
M9	Real Application, Database Restore & Lina	Complete
M10	Monitoring, Security & Governance	Complete
M11	CI/CD Quality Gates & Release Hardening	Complete
M12	Resilience, Cost Optimization & Final Validation	Complete
OP01	Automated Morning Sandbox Recovery	Implemented and validated

5. Milestones 1-3: Foundation, Containers and Availability

5.1 Milestone 1 - Foundation and Networking

Milestone 1 established the VPC and the security/route boundaries used by every later milestone. Six /24 subnets were spread across us-east-1a and us-east-1b: two public subnets, two private application subnets and two private database subnets. The public route table uses an Internet Gateway, each application subnet uses a NAT Gateway in the same Availability Zone, and the database route tables remain local-only.

Two NAT Gateways were selected instead of a single shared gateway. This increases fixed cost, but avoids making both private application subnets depend on egress through one Availability Zone. This trade-off becomes important again in the M12 cost analysis.

5.2 Milestone 2 - Container Runtime

The application was packaged with Docker and stored in Amazon ECR. Amazon ECS manages the service while AWS Fargate supplies compute without requiring EC2 server administration. Tasks run in the private application subnets with public IP assignment disabled. The initial task definition used 256 CPU units, 512 MiB memory, port 3000 and awsvpc networking. CloudWatch Logs provides application log collection.

A useful lesson from the disposable sandbox was that Terraform can recreate an ECR repository but cannot recreate the Docker image artifact that had been stored inside it. This distinction later motivated the automated application-recovery stage in OP01.

5.3 Milestone 3 - Load Balancing, Multi-AZ and Auto Scaling

The Application Load Balancer and IP target group made the private application reachable through a health-aware entry point. The service baseline was increased to two tasks, distributed across the two application subnets. Application Auto Scaling was configured with minimum capacity 2, maximum capacity 4 and a 50% target CPU policy.

A controlled validation temporarily reduced the CPU target to 1%. Generated load caused the service to scale from two to four tasks. The target was then restored to 50%. This validated behavior rather than only the Terraform configuration.

Milestones 1-3 result: A two-AZ network, private Fargate runtime, health-aware load balancing and autoscaling application baseline were working before data and edge-security services were added.

6. Milestones 4-6: CI/CD, Data, Edge Security and Authentication

6.1 Milestone 4 - Automated Deployment

GitHub Actions replaced manual container deployment. GitHub authenticates to AWS through OIDC, allowing AWS STS to issue temporary credentials to a restricted deployment role. The workflow builds the application image, tags it with the triggering Git commit SHA, pushes it to ECR, registers a new ECS task-definition revision and performs a rolling service deployment.

The deployment pipeline does not rely only on the deployment action result. Final-state checks wait for ECS service stability, verify the active image SHA, compare desired/running/pending task counts and verify the public health endpoint. This was added after an ECS deployment action once reported failure even though the service later converged successfully.

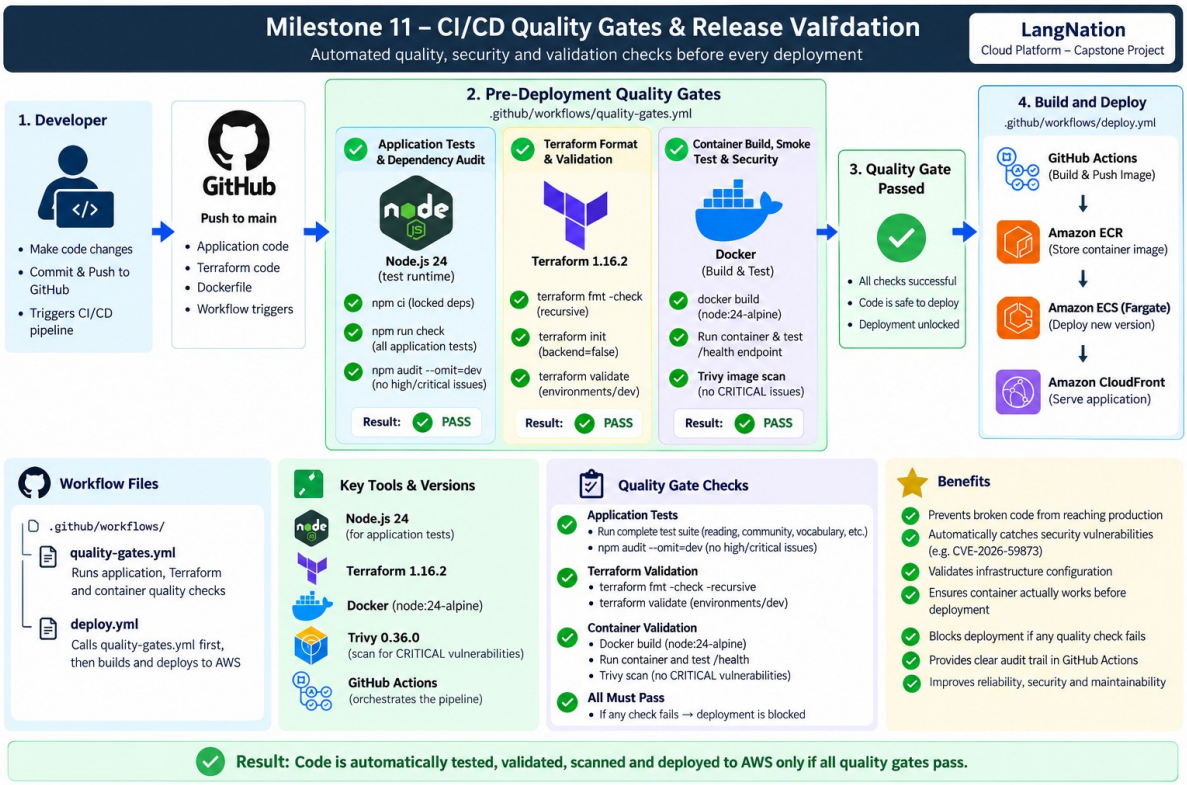


Figure 2. CI/CD deployment pipeline used to build, publish and deploy the LangNation application.

6.2 Milestone 5 - Database and Object Storage

The final data layer uses Amazon RDS for MariaDB 11.8.9 with db.t3.micro, 20 GiB gp3 storage, autoscaling up to 100 GiB, Multi-AZ deployment, encryption at rest and seven-day backups. Public access is disabled. The master password is managed by AWS Secrets Manager, and the ECS-to-database security-group path allows TCP 3306 only from the application tier.

Amazon S3 supplies private object storage. Block Public Access is enabled, object ownership is bucket-owner enforced, versioning is enabled, server-side encryption uses AES256 and bucket access requires HTTPS. Lifecycle rules limit storage growth by expiring non-current versions and incomplete multipart uploads.

6.3 Milestone 6 - Edge Security and Authentication

The public path was hardened with persistent Route 53, CloudFront, ACM and AWS WAF. Viewer traffic uses HTTPS at CloudFront. CloudFront currently reaches the ALB origin over HTTP port 80, so the project does not claim end-to-end TLS to the origin. The ALB security group allows port 80 only from the AWS-managed CloudFront origin-facing prefix list, preventing direct internet users from bypassing WAF via the ALB DNS name.

Amazon Cognito provides managed user authentication. The Node.js application validates Cognito access tokens with aws-jwt-verify. The public /health route remains available for load-balancer and deployment checks, while /protected returns HTTP 401 without a valid access token and HTTP 200 with a valid token.

M6 validation: Normal /health returned 200; a controlled suspicious request was blocked with 403; direct ALB access was blocked; /protected demonstrated 401 without JWT and 200 with a valid Cognito access token.

7. Milestone 7: Serverless and Event-Driven Processing

Milestone 7 introduced an asynchronous document-processing path. S3 Object Created events are routed through EventBridge into a durable SQS queue. Lambda polls the queue and processes messages. The queue decouples event generation from processing and makes retries explicit. A dead-letter queue preserves messages that repeatedly fail.

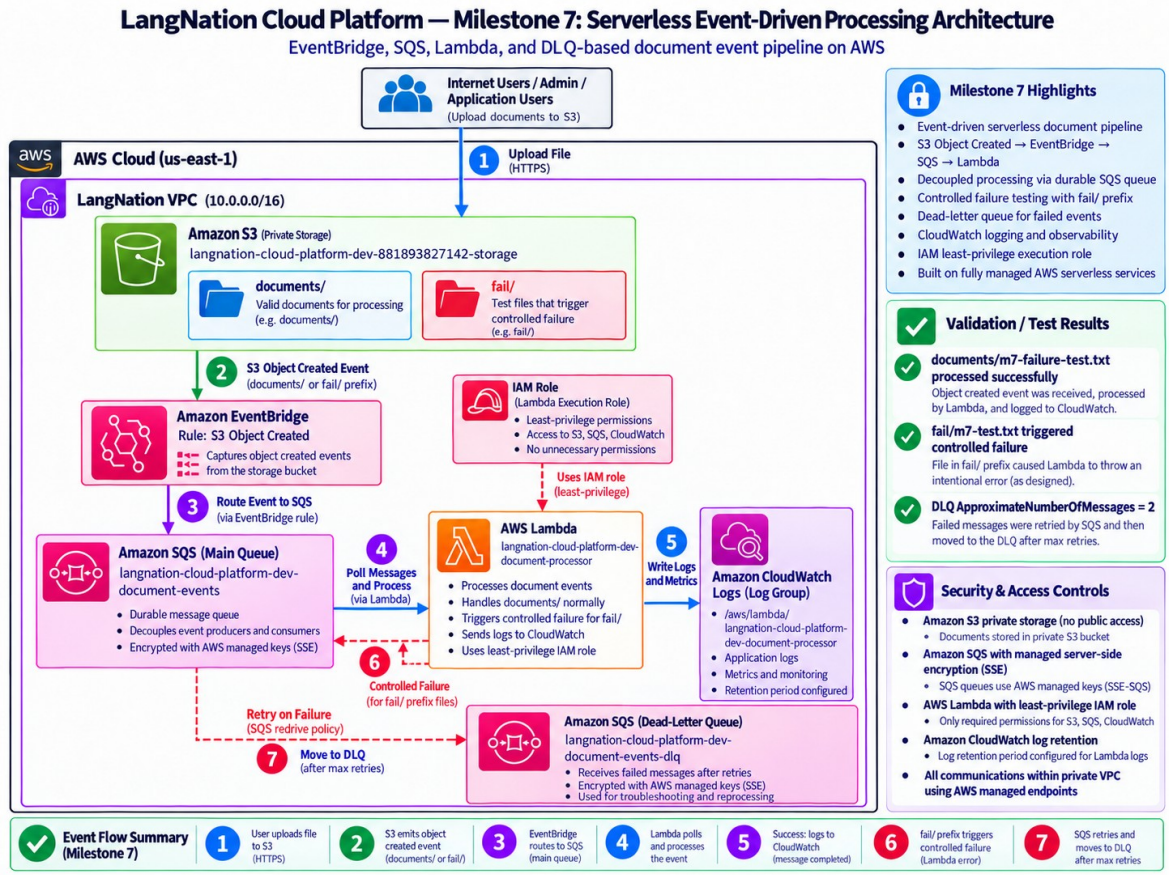


Figure 3. Milestone 7 event-driven document-processing architecture.

Table 5. M7 queue and Lambda configuration

Configuration	Value
Main queue retention	4 days
Visibility timeout	180 seconds
Encryption	SSE with AWS-managed keys
maxReceiveCount	3
DLQ retention	14 days
Lambda runtime	Python 3.12
Lambda memory	256 MB
Lambda timeout	30 seconds
Event source batch size	1
CloudWatch log retention	7 days

7.1 Controlled Failure Test

A special fail/ prefix was used to trigger an intentional RuntimeError. The same message was retried at intervals consistent with the 180-second visibility timeout and moved to the DLQ after the configured failed receives. Two DLQ messages were observed because the S3 console created both the fail/ folder marker object and the actual test object. This became an important lesson about the fact that S3 folders are key prefixes rather than real directories.

7.2 Design Outcome

M7 established a durable serverless event path that could be extended without changing the producer. Milestone 8 builds directly on this path by adding S3 GetObject, text extraction, chunking, embedding generation and vector storage.

8. Milestone 8: RAG Knowledge Pipeline

Milestone 8 converted the event processor into a knowledge-ingestion pipeline. Supported text objects are read from S3, decoded as UTF-8, chunked into approximately 200-word segments with a 40-word overlap and sent to Amazon Titan Text Embeddings V2. The returned vectors are written to Amazon S3 Vectors together with source metadata.

Table 6. M8 RAG configuration

RAG configuration	Final value
Embedding model	amazon.titan-embed-text-v2:0
Embedding dimension	1024
Vector data type	float32
Distance metric	cosine
Normalization	enabled
Chunk target	200 words
Chunk overlap	40 words
Supported ingestion	.txt, .md, .csv, .json
Index	langnation-knowledge
Encryption	AES256

8.1 Retrieval Validation

The vector index was tested with semantic questions rather than exact-string lookup. A question about German A1 articles produced the intended LangNation source chunk. In the final ranking test the relevant AWS/RAG document produced a cosine distance of approximately 0.397, while an unrelated cooking document was approximately 0.999. Because lower cosine distance represented greater similarity in the query response, the relevant document ranked first.

M8 result: The project proved document ingestion, vector creation, metadata storage, semantic query and ranking before adding an answer-generation model.

9. Milestone 9: Real Application, Database Restore and Lina RAG

Milestone 9 replaced the earlier controlled test application with a sanitized copy of the real LangNation application. This required a second recovery problem to be solved: recreating RDS infrastructure is not equivalent to recreating the application database contents. A dedicated database-restore container image and one-off ECS Fargate task were therefore added to the morning recovery path.

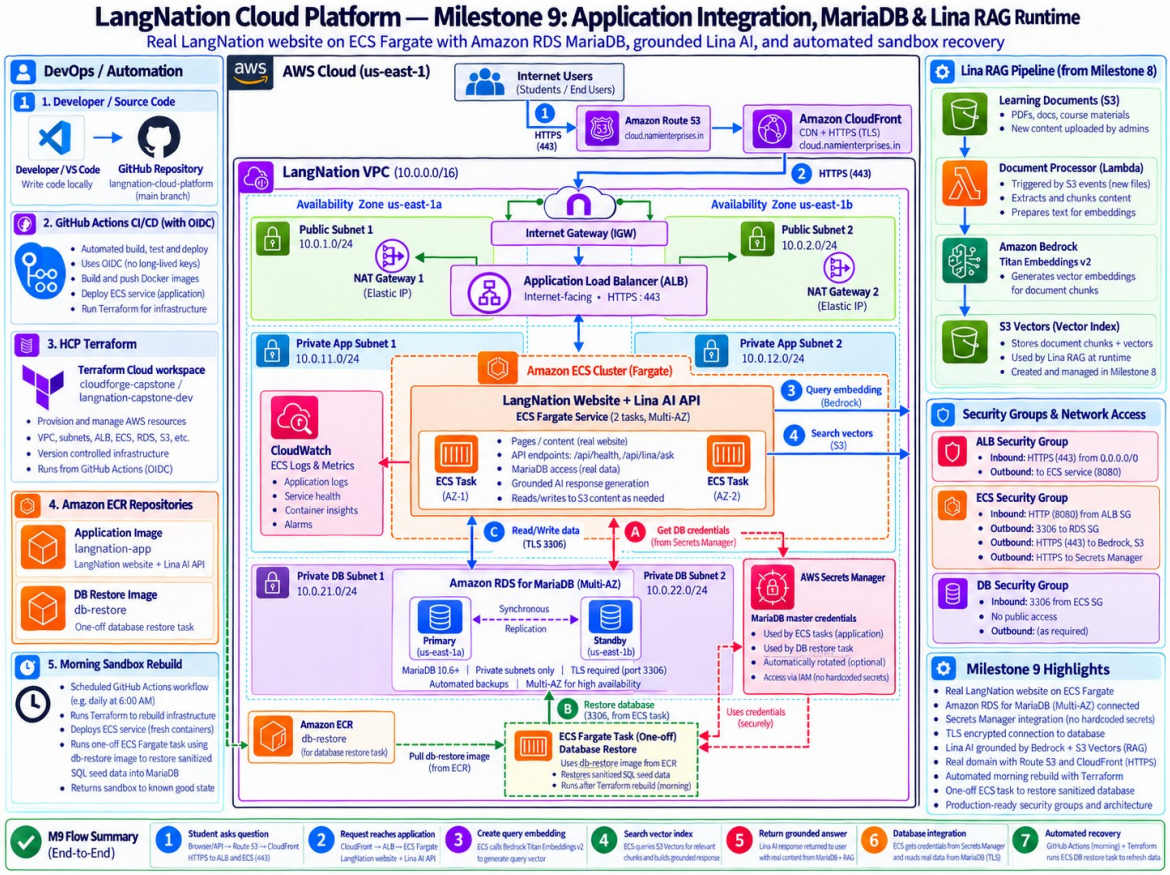


Figure 4. Milestone 9 application, database and Lina RAG integration architecture.

9.1 Sanitized Database Restore

The restore task reads the current RDS credentials from Secrets Manager, connects to the private MariaDB endpoint and imports a sanitized SQL seed. Recovery is validated by confirming the expected 58 tables. The restore task is intentionally one-off rather than a permanently running service because its purpose is environment reconstruction.

9.2 Verified TLS to MariaDB

The initial real application health check reported database=false even though the Node.js process itself was healthy. Investigation showed that the database required secure transport. The application was updated to use verified Amazon RDS TLS through aws-ssl-profiles instead of weakening the database configuration. This changed the dependency health result to database=true.

9.3 Lina Runtime RAG

```
POST /api/lina/ask
  -> validate question
  -> Titan query embedding
  -> query S3 Vectors
  -> retrieve relevant LangNation source_text
  -> invoke Claude Haiku 4.5 with retrieved context
```

```
-> return answer + grounded flag + source metadata
```

The live runtime was validated with a German A1 articles question. The API returned HTTP 200, grounded=true and the expected source key documents/german-a1-articles.txt. This is stronger evidence than proving only that the model could be invoked: it demonstrates retrieval, context injection and source attribution in the real ECS application.

M9 integration lesson: Process health and dependency health are different. A green /health endpoint did not initially reveal the database TLS failure; /api/health exposed the real dependency state.

10. Milestone 10: Monitoring, Security and Governance

M10 added centralized operations and persistent management-plane auditing. CloudWatch aggregates service metrics and exposes a single operations dashboard. Nine custom alarms cover application capacity, load-balancer health, database health, Lambda errors, DLQ backlog and CloudFront errors. CloudTrail records management events to a dedicated encrypted and versioned S3 audit bucket.

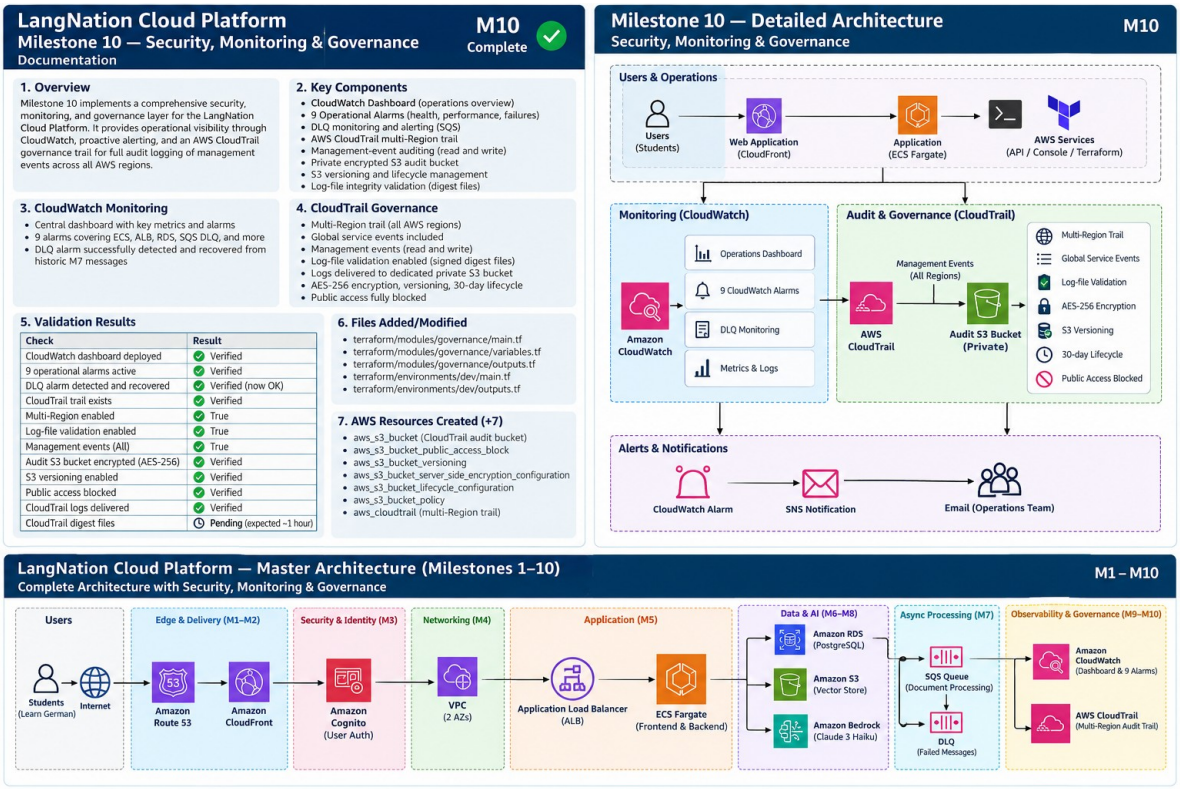


Figure 5. Milestone 10 monitoring and governance architecture.

Table 7. M10 operational alarms

Alarm	Metric	Threshold
ecs-cpu-high	ECS CPU	>= 80% for 2 x 5 min
ecs-memory-high	ECS memory	>= 80% for 2 x 5 min
alb-unhealthy-targets	ALB unhealthy hosts	>= 1 for 2 x 1 min
alb-target-5xx	ALB target 5XX	>= 5 in 5 min
rds-cpu-high	RDS CPU	>= 80% for 2 x 5 min
rds-low-storage	RDS free storage	< 2 GiB for 2 x 5 min
lambda-errors	Lambda errors	>= 1 in 5 min
document-dlq-messages	DLQ visible messages	>= 1 in 5 min
cloudfront-5xx	CloudFront 5XX rate	>= 5% for 5 x 1 min

10.1 Alarm Detection and Recovery Test

Two controlled M7 failure messages were still present in the DLQ when the M10 monitoring layer was deployed. The new DLQ alarm correctly entered ALARM. After confirming the messages were known test artifacts, the queue was purged. On the next CloudWatch evaluation the alarm returned to OK. This validated both detection and recovery behavior using a real condition.

10.2 CloudTrail Governance

The CloudTrail trail is multi-Region, includes global service events, records all management read/write events and has log-file validation enabled. Audit logs are delivered to a dedicated private S3 bucket with AES256 encryption, versioning, Block Public Access and a 30-day lifecycle suitable for the temporary training environment. A real .json.gz CloudTrail object was observed in the bucket, proving actual audit delivery.

11. Milestone 11: CI/CD Quality Gates and Release Hardening

M11 changed the deployment pipeline from simple automation into a controlled release process. A reusable GitHub Actions workflow now runs mandatory checks before the build-and-deploy stage is allowed to execute.

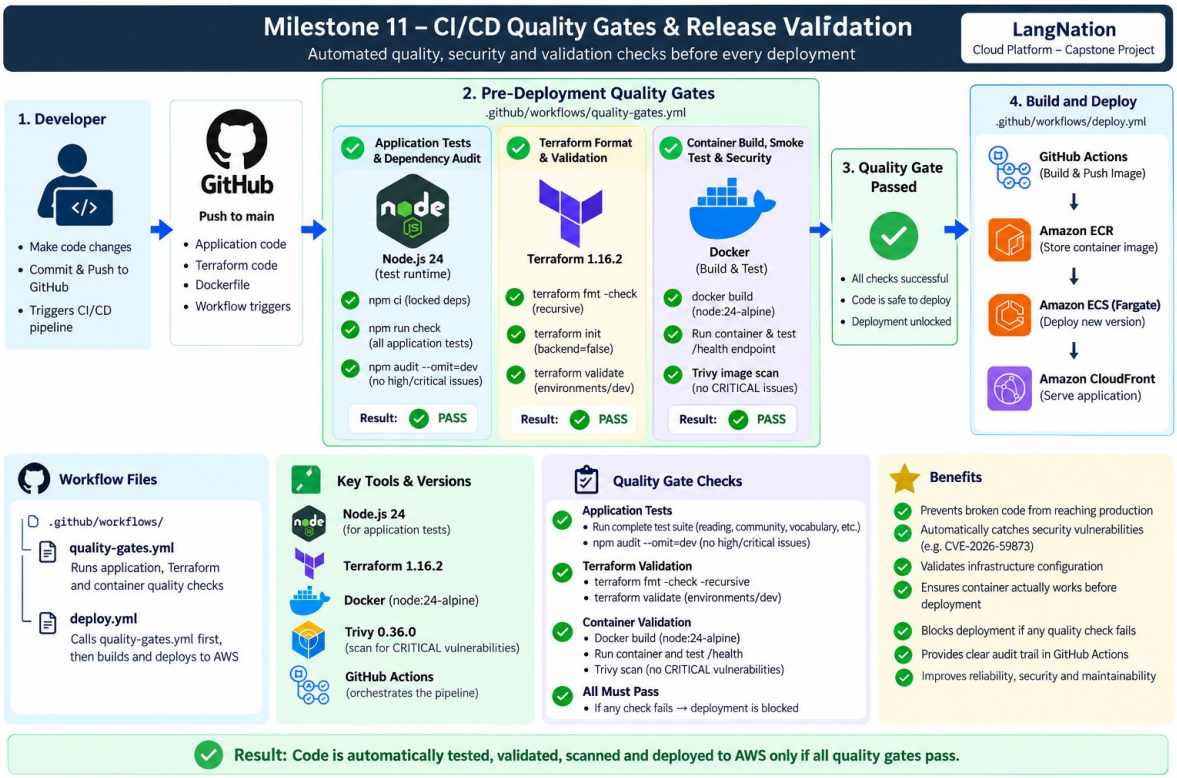


Figure 6. Milestone 11 CI/CD quality-gate architecture.

11.1 Quality-Gate Sequence

1. Application dependency installation with npm ci.
2. Application check using Node.js 24.
3. npm audit at the configured high-severity threshold.
4. Terraform formatting check, backend-free initialization and terraform validate using Terraform 1.16.2.
5. Production Docker image build.
6. Container startup and /health smoke test.
7. Trivy container vulnerability scan with CRITICAL findings configured to fail the workflow.
8. Final quality-gate decision. Only a successful result permits deployment.

11.2 Real Security Failure and Remediation

Trivy detected a CRITICAL vulnerability in the previous node:20-alpine production image, involving the bundled tar 6.2.1 package (CVE-2026-59873). The project did not disable the scan, lower the severity threshold or whitelist the finding. Instead, the production base image was upgraded to node:24-alpine and the full workflow was rerun.

The final successful release used commit caaa14a84120191835d74a57c16e5cf23543d449 with the message "fix: upgrade production container to Node 24 LTS". The final Deploy run completed successfully with all quality-gate jobs green before the ECS deployment.

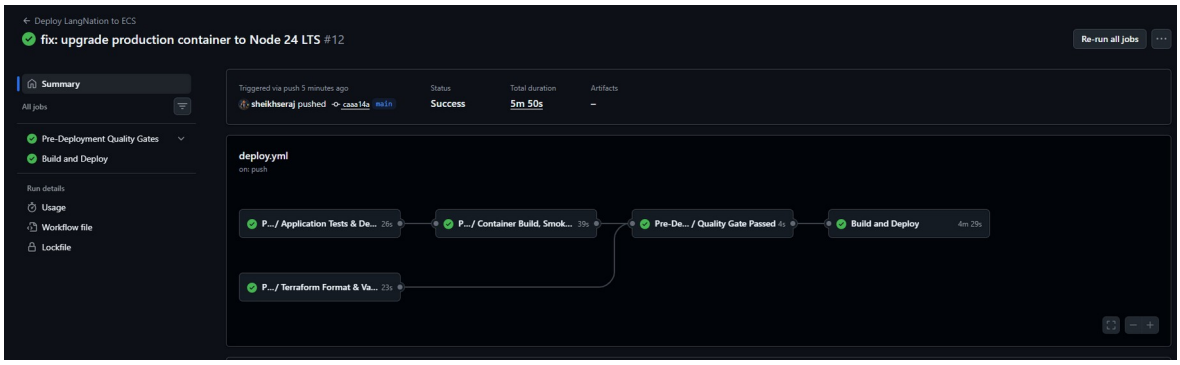


Figure 7. Evidence of the successful M11 quality-gated deployment.

Release principle: A security gate provides value only if it is allowed to stop the release and the root cause is fixed rather than bypassed.

12. Milestone 12: Resilience, Cost Optimization and Final Validation

12.1 Resilience Baseline

Before introducing failure, the ECS service was observed at desired=2, running=2 and pending=0, with tasks distributed across us-east-1a and us-east-1b. The public health endpoint returned HTTP 200. This established the baseline before the controlled task failure.

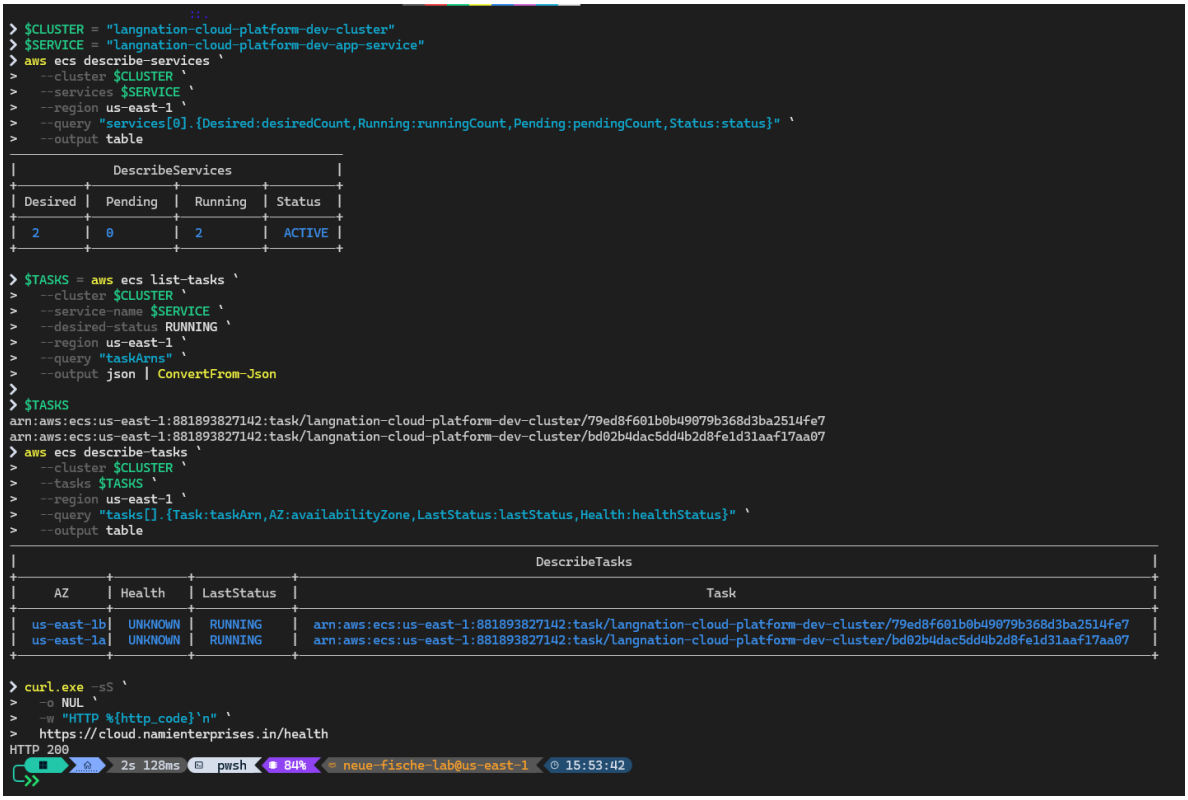


Figure 8. M12 resilience baseline with two tasks across two Availability Zones and HTTP 200.

12.2 Controlled Task Failure

One running ECS service task was intentionally stopped. The service immediately moved to desired=2, running=1 and pending=1 while the public health endpoint continued returning HTTP 200. This demonstrated that the surviving task and ALB health-aware routing maintained availability during the replacement process.

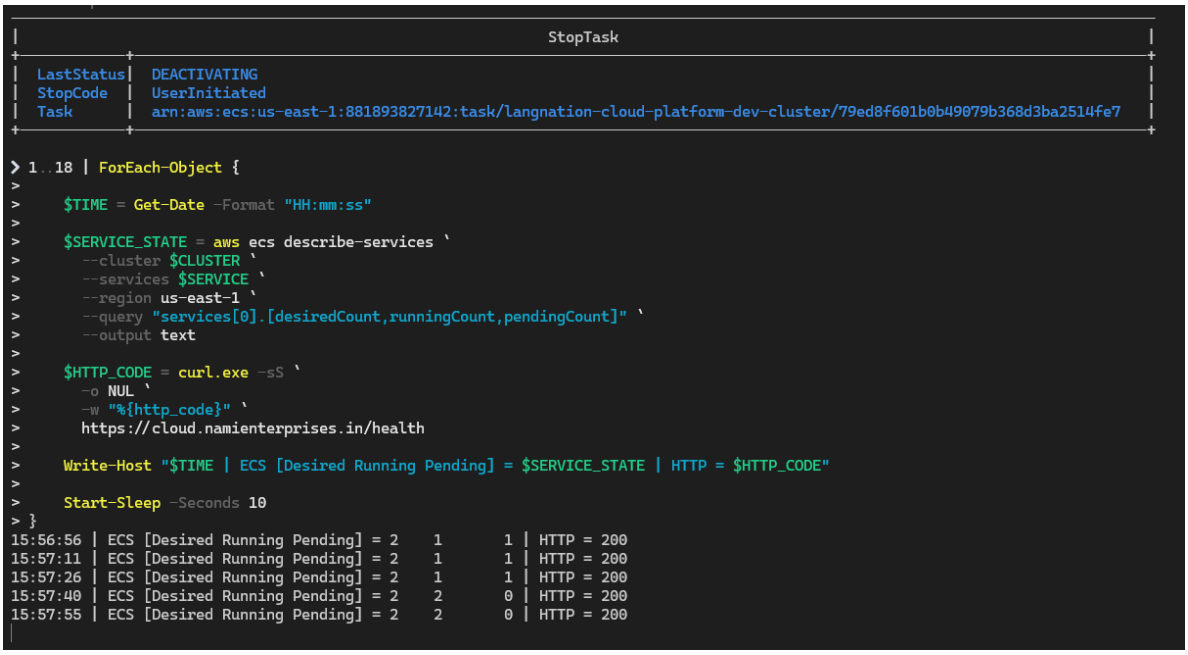


Figure 9. M12 live self-healing observation during the controlled ECS task failure.

ECS automatically launched a replacement task and restored the service to desired=2, running=2 and pending=0 in approximately 44 seconds. No manual replacement task was started.

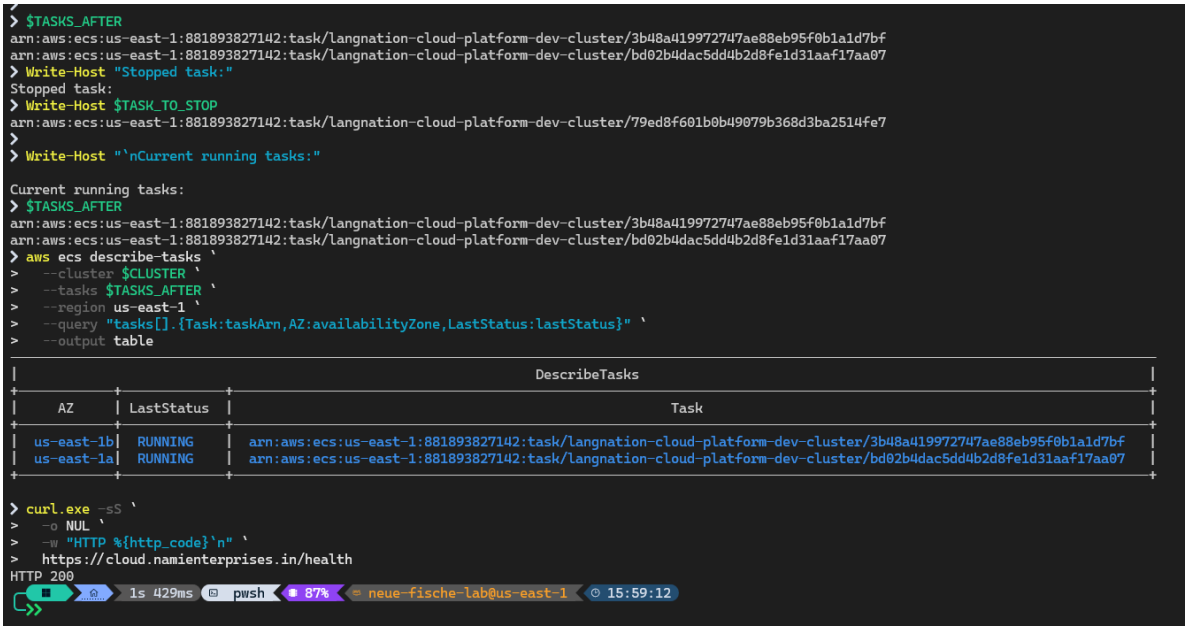


Figure 10. M12 replacement task final state after automatic recovery.

12.3 Cost Explorer Analysis

AWS Cost Explorer was reviewed for 1-23 October 2026. The captured usage subtotal before tax was approximately $23.78. The displayed tax was $4.29, producing a total of approximately $28.07 including the shown tax. The largest service category was EC2 - Other, which required deeper usage-type inspection because the application runtime itself used Fargate rather than EC2 worker instances.

Table 8. M12 service-level cost breakdown (captured period)

Service	Captured cost (USD)
EC2 - Other	$12.2177
Amazon RDS	$3.1456

Service	Captured cost (USD)
Amazon ECS	$2.3248
Elastic Load Balancing	$2.2282
Amazon VPC	$2.1566
AWS WAF	$0.8738
Amazon CloudWatch	$0.6121
AWS Service Catalog	$0.1680
AWS Secrets Manager	$0.0422
Amazon S3	$0.0113
Amazon ECR	$0.0009
AWS KMS	$0.0005

GetCostAndUsage

Cost	Service	Unit
0.000531	AWS Key Management Service	USD
0.0422199075	AWS Secrets Manager	USD
0.168	AWS Service Catalog	USD
0.0000001386	AWS Step Functions	USD
0.8738488541	AWS WAF	USD
0.0000115	Amazon Bedrock	USD
0.0000009408	Amazon CloudFront	USD
0.0008852405	Amazon EC2 Container Registry (ECR)	USD
12.2177053311	EC2 - Other	USD
2.3248060197	Amazon Elastic Container Service	USD
2.228246535	Amazon Elastic Load Balancing	USD
0.0000001134	Amazon Glacier	USD
3.1456372965	Amazon Relational Database Service	USD
0.0113097274	Amazon Simple Storage Service	USD
2.156584765	Amazon Virtual Private Cloud	USD
0.612077321	AmazonCloudWatch	USD
0.000012	CloudWatch Events	USD
4.29	Tax	USD

Figure 11. M12 AWS Cost Explorer service breakdown evidence.

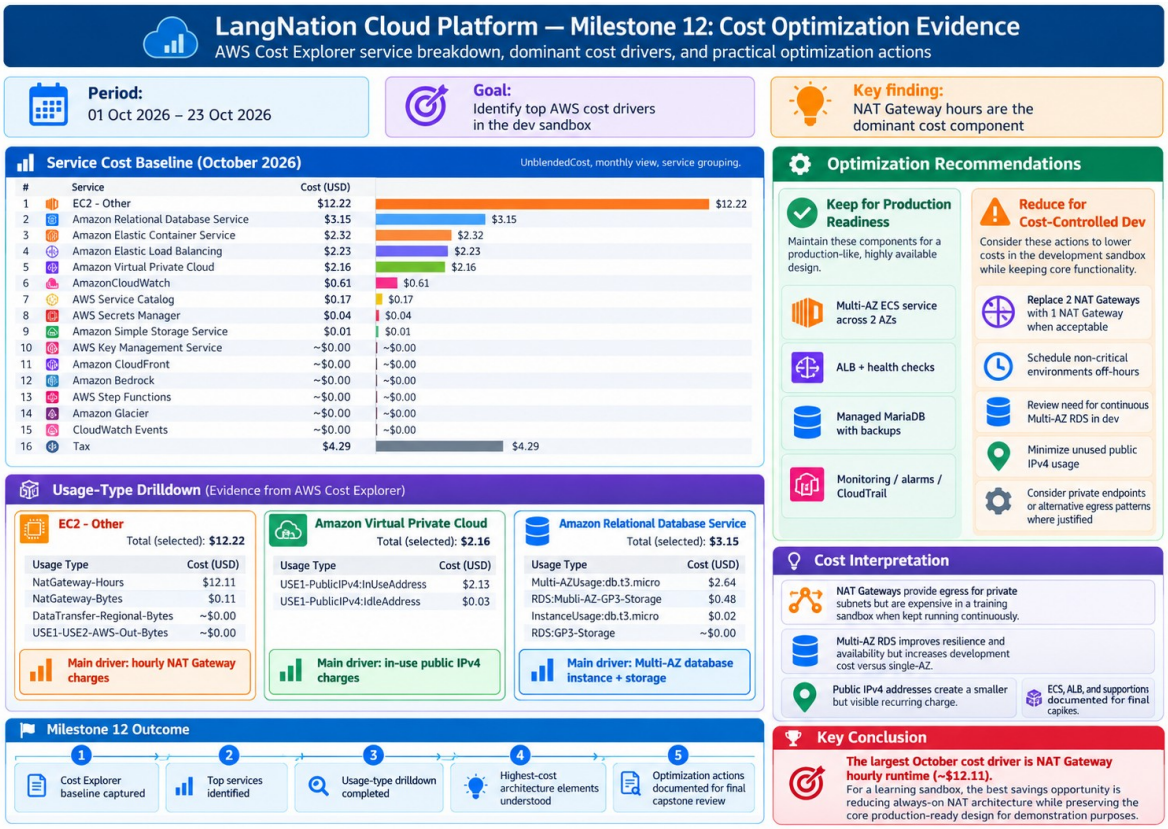


Figure 12. M12 cost-optimization summary and design trade-offs.

12.4 Final Cost Decision

Detailed usage analysis showed NAT Gateway hourly usage as an important fixed-cost driver. Multi-AZ RDS, two baseline Fargate tasks and the ALB also contribute continuing cost. Lower-cost development alternatives include one NAT Gateway, Single-AZ RDS, scheduled non-production runtime and tighter public IPv4 use. These changes were documented but not applied to the final capstone because they would weaken the availability and resilience evidence that the project was designed to demonstrate.

13. OP01: Automated Morning Sandbox Recovery

The educational sandbox removes workload resources overnight. The neue-fische sandbox itself must be activated manually because GitHub Actions cannot start the educational lab session. Once an active AWS session exists, OP01 coordinates the remaining recovery as one dependency-driven workflow.

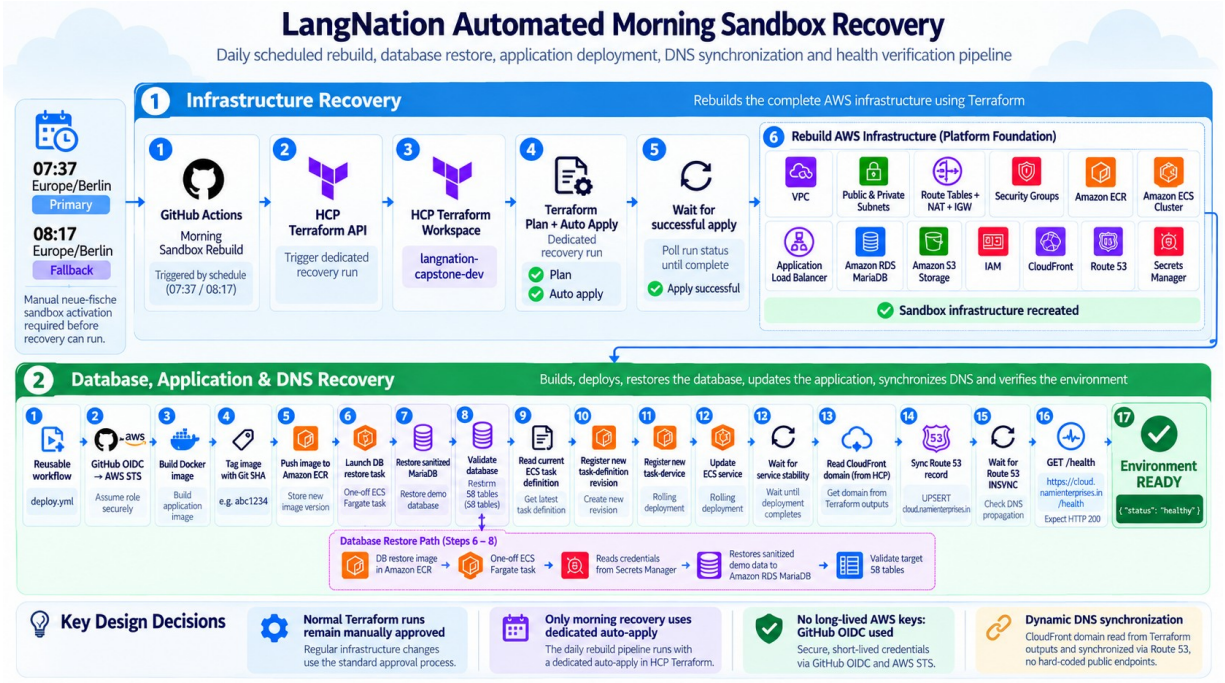


Figure 13. Automated morning sandbox recovery flow.

13.1 Recovery Sequence

1. Primary schedule at 07:37 Europe/Berlin or manual workflow_dispatch starts the recovery workflow; 08:17 acts as a fallback.
2. GitHub Actions calls the HCP Terraform API and creates a dedicated recovery run with auto-apply requested for that run only.
3. The workflow polls HCP Terraform until infrastructure recreation succeeds.
4. A database-restore image is built and pushed to ECR.
5. A one-off ECS Fargate restore task reads the current RDS credentials from Secrets Manager, restores the sanitized MariaDB seed and validates 58 tables.
6. The reusable application deployment workflow builds the Git-SHA application image, pushes it to ECR, registers a new task-definition revision, updates ECS and waits for service stability.
7. The workflow reads the recreated CloudFront domain from HCP Terraform outputs.
8. GitHub assumes a restricted Route 53 role in the persistent personal AWS account through OIDC.
9. The cloud.namienterprises.in record is updated, the workflow waits for Route 53 INSYNC and then validates the public HTTPS /health endpoint.

13.2 Recovery Design Principles

- Normal Terraform runs remain manually approved; automatic apply is isolated to the known morning recovery path.
- Recovery is chained by successful completion, not by independent timers, avoiding race conditions between infrastructure and application restoration.
- No long-lived AWS deployment keys are stored in GitHub.
- DNS is persistent while CloudFront and other workload resources are disposable.
- The workflow validates the same protected public path used by real clients instead of bypassing WAF with a direct ALB check.

14. Security Architecture and Threat Reduction

Security decisions were introduced throughout the milestones rather than added as one final layer. The final architecture reduces direct exposure, uses short-lived identities for automation, keeps secrets out of source code, separates application and data tiers and validates public requests through a protected edge.

Table 9. Security-control mapping

Control	Implementation	Risk reduced
Network segmentation	Public/app/DB subnets; DB route tables local-only	Limits blast radius and internet exposure
Private ECS	No public IP; ingress from ALB only	Prevents direct container access
Private RDS	Public access disabled; DB SG from ECS only	Prevents direct database access
CloudFront-only ALB	Origin-facing prefix-list ingress	Prevents WAF bypass via ALB DNS
WAF	Managed common rule set + rate rule	Blocks common web attacks/abuse
TLS	HTTPS viewer edge; verified RDS TLS	Protects client traffic and DB transport
Cognito/JWT	Managed authentication and token validation	Protects authenticated API routes
GitHub OIDC	Short-lived STS credentials	Eliminates long-lived CI/CD AWS keys
Secrets Manager	Managed database password	Removes DB password from repository/workflows
SQS/DLQ	Durable failure handling	Prevents uncontrolled retry loops and preserves failures
CloudTrail	Persistent management audit	Provides governance evidence
Trivy quality gate	CRITICAL findings block release	Reduces deployment of known vulnerable images

14.1 Least Privilege

The project uses distinct IAM roles for ECS task execution, application tasks, GitHub deployment and personal-account Route 53 updates. EventBridge is restricted to sending messages to the intended queue, Lambda permissions are scoped to the resources required for processing, and the CloudTrail S3 bucket policy permits only the delivery actions needed by the trail.

14.2 Security Limitations

The current CloudFront-to-ALB origin path uses HTTP port 80. Viewer HTTPS is protected by ACM at CloudFront, and direct ALB access is blocked. The project therefore provides a protected public edge but does not claim encrypted TLS transport from CloudFront to the ALB origin. A production hardening path could add HTTPS at the ALB origin if required.

15. Testing, Validation and Evidence

Validation was treated as a design requirement. Terraform resource creation was not considered sufficient evidence for behavior. Each milestone included runtime checks, controlled failures or security tests appropriate to the feature being introduced.

Table 10. Final validation matrix

Area	Validation focus	Evidence	Result
M1	Network/IaC	HCP Terraform rebuild and correct subnet/routing design	PASS
M2	Container runtime	Private Fargate task and /health runtime	PASS
M3	Availability/scaling	Two-AZ baseline; controlled 2 -> 4 scale test	PASS
M4	CI/CD	OIDC release, SHA traceability, stability/count/health checks	PASS
M5	Data	Private encrypted MariaDB/S3 configuration	PASS
M6	Edge/auth	200 health, 403 WAF, ALB bypass blocked, JWT 401/200	PASS
M7	Events	Success path, 180 s retries, DLQ delivery	PASS
M8	RAG retrieval	Embedding/vector write, semantic retrieval and ranking	PASS
M9	Integration	58-table restore, verified DB TLS, grounded Lina response	PASS
M10	Operations	Dashboard, 9 alarms, DLQ ALARM -> OK, CloudTrail object	PASS
M11	Release quality	Tests, Terraform gate, smoke test, Trivy remediation	PASS
M12	Resilience/cost	HTTP 200 during task failure, ~44 s recovery, cost analysis	PASS
OP01	Recovery	Rebuild + DB restore + deploy + DNS sync + public health	PASS

15.1 Evidence Organization

The repository separates explanatory milestone documents from validation evidence. Milestone documentation explains what was built, why it was designed that way and what was learned. Evidence files record PASS/FAIL details, exact controlled tests and screenshots. This separation prevents the main technical narrative from becoming a raw log while preserving proof for review.

15.2 Evidence Quality

- Prefer behavior evidence such as HTTP 200, ALARM -> OK, 2/1/1 -> 2/2/0 and grounded=true over screenshots that only show that a resource exists.
- Use controlled failure tests where possible.
- Keep secrets, access tokens, private keys and real personal data out of committed screenshots.
- Keep architecture diagrams separate from operational evidence screenshots.

16. Cost Analysis and Optimization Decisions

Cost was evaluated using actual Cost Explorer data from the development period rather than relying only on monthly price estimates. The captured spend reflects the temporary sandbox usage pattern and should not be interpreted as a production monthly forecast.

16.1 Main Cost Drivers

The service breakdown showed EC2 - Other as the largest category, followed by RDS, ECS, Elastic Load Balancing and VPC. Usage-type inspection indicated that NAT Gateway hourly availability was an important fixed networking cost even though the processed byte volume was low. This illustrates why cost analysis should continue beyond a high-level service label.

16.2 Optimization Options Considered

- Use a single NAT Gateway in a development-only environment.
- Use Single-AZ RDS outside resilience demonstrations.
- Schedule non-production runtime where service constraints permit.
- Reduce unnecessary public IPv4 use.
- Evaluate VPC endpoints or alternative egress only after comparing their hourly and data-processing costs against NAT for the actual traffic pattern.

16.3 Why the Final Architecture Was Not Simplified

The final project deliberately keeps two NAT Gateways, two baseline ECS tasks, Multi-AZ RDS and the ALB. Removing those components would reduce cost but would also weaken the high-availability architecture and invalidate parts of the final resilience evidence. The project therefore separates production-style architecture decisions from development optimization opportunities.

17. Discussion, Limitations and Lessons Learned

17.1 Major Engineering Challenges

Table 11. Major technical challenges and permanent fixes

Challenge	Root cause / context	Permanent solution
Nightly sandbox cleanup	AWS resources and runtime artifacts disappear	Treat environment as reproducible; HCP Terraform + OP01 recovery
Local Terraform state mismatch	Local plan did not reflect authoritative remote state	Use local init/fmt/validate only; HCP Terraform for plan/apply/state
OIDC trust failure	GitHub identity conditions did not match actual immutable subject	Restrict trust to immutable repository/owner IDs and main branch
ECS deploy action false failure	Transient rollout state interpreted as final failure	Independent services-stable, SHA, count and health checks
ALB bypass risk	Internet users could reach origin directly	Move health validation to CloudFront and restrict ALB ingress
RDS secure transport	Real app DB health false	Use verified RDS TLS instead of weakening database security
DB restore credentials	Generated password contained shell-special character	Treat credentials as arbitrary strings and validate connection before import
Two DLQ messages	S3 console created folder marker + file object	Understand S3 key-prefix behavior
Trivy CRITICAL finding	Old Node base image bundled vulnerable tar package	Upgrade to Node 24; keep security threshold intact
Cost vs resilience	Cheapest architecture would remove redundancy	Document dev optimizations but preserve final HA design

17.2 Key Lessons

- Infrastructure reproducibility must include runtime artifacts, data restoration and external DNS dependencies - not only Terraform resources.
- High availability should be demonstrated through controlled failure and recovery, not inferred from a diagram.
- One system should own each live runtime value to avoid configuration drift and automation conflict.
- A public process health endpoint is insufficient for complex applications; dependency health must also be observable.
- RAG quality depends on ingestion, chunking, embedding consistency, metadata and retrieval validation before generation is added.
- Monitoring is most useful when it detects a real condition and visibly returns to healthy after remediation.
- Security gates must be allowed to fail the release; otherwise they are reporting tools, not controls.
- Cost optimization should be evaluated against the purpose of the environment and the architecture requirements being demonstrated.

17.3 Limitations

- The environment is a development/training sandbox rather than a production account.
- Traffic volumes are validation-scale rather than real production traffic.
- CloudFront-to-ALB transport uses HTTP port 80 in the current architecture.
- The project demonstrates AWS management-event auditing but not full data-event auditing or Security Hub/GuardDuty deployment.
- The RAG corpus is intentionally small for a capstone; retrieval at large knowledge-base scale was not tested.
- The cost analysis reflects a partial October training period, not a full production month.

18. Conclusion and Future Work

18.1 Conclusion

The LangNation Cloud Platform capstone evolved from a basic network into an integrated AWS application platform with secure edge delivery, private compute and data, event-driven processing, retrieval-augmented generation, automated deployment, monitoring, governance, quality gates, resilience validation and recovery automation. The milestone approach kept the architecture understandable while allowing each new capability to build on previously validated foundations.

The strongest outcome is not the number of AWS services used. It is the fact that the services were connected through explicit traffic paths and ownership rules and then validated with real behavior. The project demonstrates that the environment can be rebuilt from code, the application and database can be restored, the public DNS can follow recreated infrastructure, a vulnerable container can be blocked before release, an event can be retried and dead-lettered, an AI answer can be grounded in retrieved LangNation content, and an ECS task can fail without taking down the public endpoint.

The final architecture therefore meets the capstone goal of a secure, highly available, containerized and AI-enabled learning platform that is reproducible under an ephemeral AWS environment constraint.

18.2 Future Work

- Create separate Terraform variable profiles for production-style and lower-cost development environments.
- Add HTTPS encryption between CloudFront and the ALB origin if production requirements demand end-to-end TLS.
- Expand the LangNation knowledge corpus and evaluate retrieval quality with a larger benchmark set.
- Add application-level tracing and richer SLO/SLA dashboards where useful.
- Introduce a formal deployment rollback runbook and automated rollback policy if release frequency increases.
- Extend database recovery testing to timed restore objectives and documented RPO/RTO targets.
- Add stronger production governance services only when their operational value and cost are justified.

References

[1] Amazon Web Services, AWS Well-Architected Framework.

[2] Amazon Web Services, Amazon VPC User Guide.

[3] Amazon Web Services, Amazon ECS Developer Guide and AWS Fargate documentation.

[4] Amazon Web Services, Elastic Load Balancing - Application Load Balancers.

[5] Amazon Web Services, Amazon RDS for MariaDB documentation.

[6] Amazon Web Services, Amazon S3 User Guide.

[7] Amazon Web Services, Amazon CloudFront Developer Guide.

[8] Amazon Web Services, AWS WAF Developer Guide.

[9] Amazon Web Services, Amazon Cognito Developer Guide.

[10] Amazon Web Services, Amazon EventBridge User Guide.

[11] Amazon Web Services, Amazon SQS Developer Guide.

[12] Amazon Web Services, AWS Lambda Developer Guide.

[13] Amazon Web Services, Amazon Bedrock User Guide.

[14] Amazon Web Services, Amazon S3 Vectors documentation.

[15] Amazon Web Services, Amazon CloudWatch User Guide.

[16] Amazon Web Services, AWS CloudTrail User Guide.

[17] HashiCorp, Terraform documentation and HCP Terraform documentation.

[18] GitHub, GitHub Actions and OpenID Connect in AWS documentation.

[19] Aqua Security, Trivy documentation.

[20] Internal project documentation: LangNation Cloud Platform Milestones M01-M12, validation evidence and OP01 operations runbook, October 2026.

Appendix A - Final Technical Configuration

Table 12. Final technical configuration

Configuration	Final value
AWS Region	us-east-1
VPC	10.0.0.0/16
Public subnets	10.0.1.0/24, 10.0.2.0/24
Private app subnets	10.0.11.0/24, 10.0.12.0/24
Private DB subnets	10.0.21.0/24, 10.0.22.0/24
NAT Gateways	2, one per AZ
ECS task CPU / memory	256 CPU / 512 MiB
Container port	3000
ECS baseline	desired 2
ECS autoscaling	min 2, max 4, target CPU 50%
ALB health path	/health
RDS engine	MariaDB 11.8.9
RDS class	db.t3.micro
RDS storage	20 GiB gp3, max 100 GiB
RDS availability	Multi-AZ
RDS backups	7 days
S3 storage	Private, versioned, AES256, HTTPS-only
Event queue visibility	180 seconds
SQS maxReceiveCount	3
DLQ retention	14 days
Lambda runtime	Python 3.12
Lambda memory / timeout	256 MB / 30 s
Embedding model	amazon.titan-embed-text-v2:0
Embedding dimension	1024
Chunk target / overlap	200 / 40 words
CloudWatch custom alarms	9
CloudTrail	Multi-Region; management events; validation enabled
Public hostname	cloud.namienterprises.in
Primary recovery schedule	07:37 Europe/Berlin
Fallback recovery schedule	08:17 Europe/Berlin

Appendix B - Repository Structure

```
LangNation-Cloud-Platform-Capstone/
|-- .github/workflows/
|   |-- quality-gates.yml
|   |-- deploy.yml
|   `-- morning-rebuild.yml
|-- application/
|-- database/
|-- knowledge/
|-- lambda/
|-- terraform/
|   |-- environments/dev/
|   `-- modules/
|       |-- networking/
|       |-- security/
|       |-- container/
|       |-- load_balancing/
|       |-- database/
|       |-- storage/
|       |-- authentication/
|       |-- event_processing/
|       |-- rag_knowledge/
|       |-- monitoring/
|       `-- governance/
`-- docs/
    |-- milestones/
    |-- architecture/
    |-- evidence/
    |-- operations/
    `-- presentation/
```

The repository separates application code, infrastructure modules, knowledge source content, event-processing code and documentation. HCP Terraform remains the authoritative infrastructure state system, so generated local state and disposable artifacts are excluded from the final repository.

Appendix C - Evidence and Documentation Map

Artifact	Location	Purpose
Milestone documentation	docs/milestones/M01...M12	Architecture, implementation, validation, problems, lessons
Architecture diagrams	docs/architecture/milestones/	Design sketch + milestone + cumulative platform images
Cross-cutting flows	docs/architecture/flows/	CI/CD and morning recovery
Final master architecture	docs/architecture/master/final-platform-m01-m12.png	Complete final platform
Validation evidence	docs/evidence/	Selective detailed proof and screenshots
Operations	docs/operations/OP01-automated-morning-sandbox-recovery.md	Repeatable recovery runbook
Presentation	final capstone deck	Presentation-ready M1-M12 summary

Appendix D - Final Validation Checklist

Validation item	Result
VPC/subnets/routing across two AZs	PASS
Private ECS tasks	PASS
ALB health-aware routing	PASS
Auto Scaling 2-4	PASS
OIDC CI/CD	PASS
Private Multi-AZ MariaDB	PASS
S3 private/versioned/encrypted	PASS
CloudFront/ACM/WAF edge	PASS
Cognito JWT auth	PASS
EventBridge/SQS/Lambda/DLQ	PASS
Titan embeddings + S3 Vectors	PASS
Lina grounded RAG	PASS
CloudWatch dashboard + 9 alarms	PASS
CloudTrail audit delivery	PASS
CI/CD quality gates + Trivy	PASS
ECS controlled self-healing test	PASS
AWS cost analysis	PASS
OP01 morning recovery	PASS

Appendix E - Submission Summary

This report is intended to accompany the final source repository, architecture documentation, evidence screenshots and final presentation. Together, these artifacts provide four complementary views of the capstone: the implementation itself, the technical narrative, the operational proof and the presentation summary.

Submission package: Final report (DOCX/PDF) + final M1-M12 presentation + GitHub repository + architecture/evidence/operations documentation.

End of report.